



Esercizio EpiCode

Attacchi alle Web App2

ESERCIZIO EPICODE WEEK 13 DAY 4 A CURA DI FABIO RENNA

Indice

Executive Summary.....	2
Introduzione.....	2
Obiettivo.....	3
Proof of Concept.....	3 -10
Conclusione.....	10

Executive Summary

Nel corso dell'attività di laboratorio sono state analizzate e sfruttate con successo due vulnerabilità web critiche presenti nell'applicazione **DVWA (Damn Vulnerable Web Application)**, configurata con livello di sicurezza **LOW: XSS Reflected** e **SQL Injection non blind**.

Attraverso tecniche di **Cross-Site Scripting**, è stato possibile eseguire codice JavaScript arbitrario nel browser della vittima, dimostrando il potenziale rischio di furto di cookie di sessione e compromissione dell'account.

Parallelamente, mediante una **SQL Injection non blind**, è stato possibile manipolare le query SQL lato server ed estrarre dati sensibili dal database, inclusi nomi utente e hash delle password.

L'attività evidenzia come la mancata validazione dell'input utente e l'assenza di meccanismi di protezione adeguati possano portare a gravi impatti sulla **confidenzialità, integrità e sicurezza complessiva** dell'applicazione web.

Introduzione

L'esercizio è stato svolto all'interno di un laboratorio virtuale composto da una macchina **Kali Linux** (attaccante) e una macchina **Metasploitable** (target), sulla quale è installata l'applicazione vulnerabile **DVWA**.

Il traffico di rete e le interazioni con l'applicazione sono state effettuate tramite browser e analizzate quando necessario.

DVWA è una piattaforma didattica progettata per simulare vulnerabilità web comuni, permettendo di comprendere sia il funzionamento degli attacchi sia i rischi associati a una configurazione non sicura delle applicazioni web.

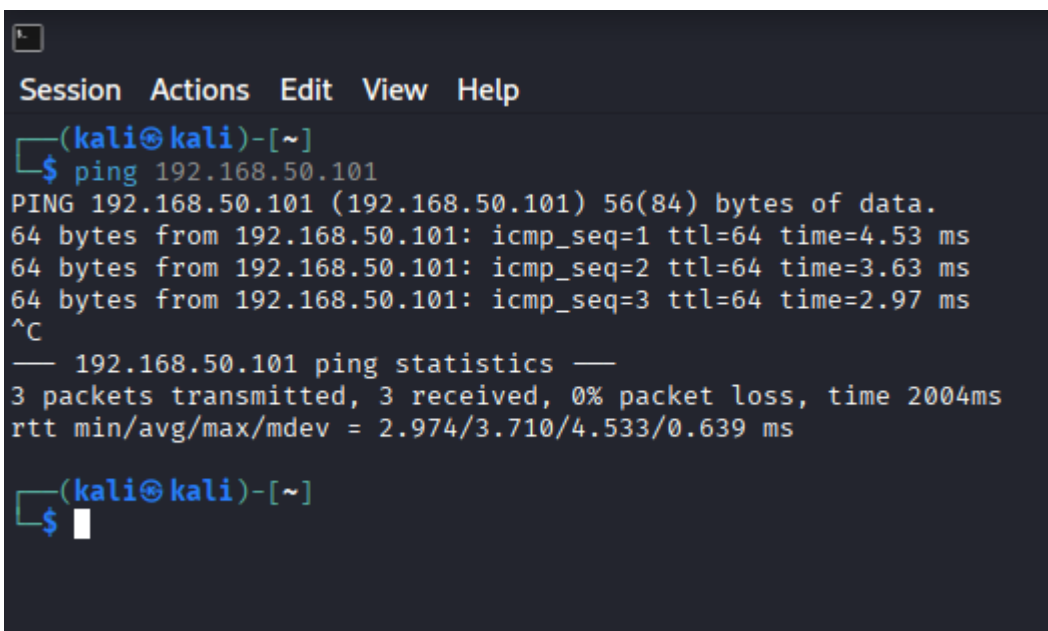
In questo contesto, il laboratorio si focalizza su due delle vulnerabilità più diffuse e pericolose nel mondo reale: **Cross-Site Scripting (XSS)** e **SQL Injection**.

Obiettivo

L'obiettivo dell'esercizio è:

- verificare la corretta configurazione del laboratorio virtuale e la comunicazione tra le macchine
- raggiungere l'applicazione DVWA dalla macchina attaccante
- impostare il livello di sicurezza dell'applicazione su **LOW**
- sfruttare una vulnerabilità di **XSS Reflected** per eseguire codice lato client
- sfruttare una vulnerabilità di **SQL Injection non blind** per manipolare query SQL ed estrarre dati dal database
- documentare ogni fase dell'attacco tramite **screenshot e spiegazioni tecniche** in un report strutturato

Proof of Concept



```
Session  Actions  Edit  View  Help
(kali㉿kali)-[~]
$ ping 192.168.50.101
PING 192.168.50.101 (192.168.50.101) 56(84) bytes of data:
64 bytes from 192.168.50.101: icmp_seq=1 ttl=64 time=4.53 ms
64 bytes from 192.168.50.101: icmp_seq=2 ttl=64 time=3.63 ms
64 bytes from 192.168.50.101: icmp_seq=3 ttl=64 time=2.97 ms
^C
— 192.168.50.101 ping statistics —
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 2.974/3.710/4.533/0.639 ms
(kali㉿kali)-[~]
$
```

Prima di procedere con lo sfruttamento delle vulnerabilità XSS e SQL Injection, è stata verificata la corretta comunicazione di rete tra la macchina attaccante (**Kali Linux**) e la macchina target (**Metasploitable**).

Dal terminale di Kali Linux è stato eseguito il comando:

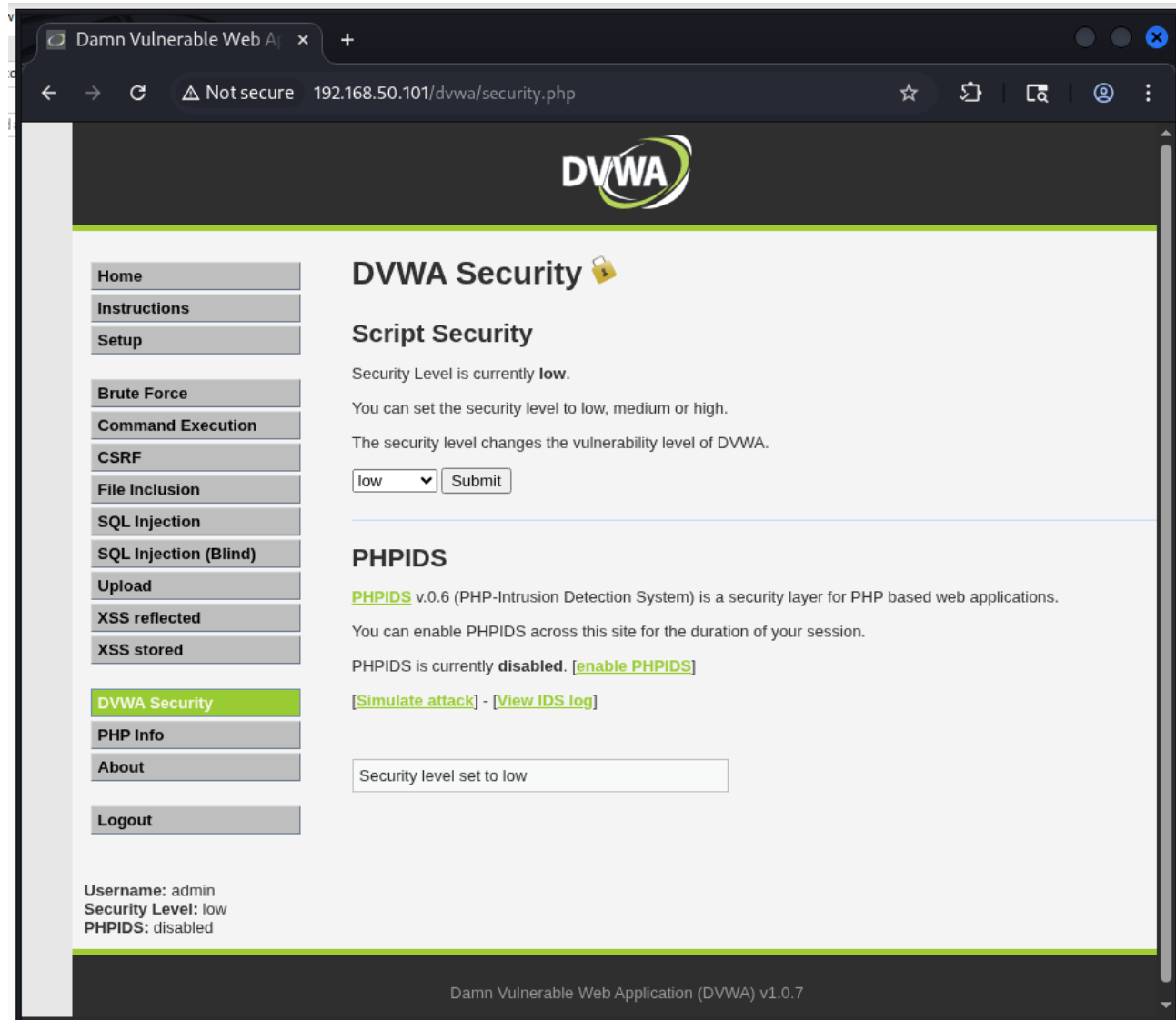
```
ping 192.168.50.101
```

Risultato:

La risposta positiva ai pacchetti ICMP conferma che:

- le due macchine sono correttamente configurate sulla stessa rete
- la macchina target è raggiungibile dall'attaccante
- il laboratorio virtuale è pronto per le successive fasi di test

Questa verifica preliminare è fondamentale per garantire che eventuali problemi riscontrati nelle fasi successive non siano dovuti a errori di configurazione di rete.



Dopo aver verificato la corretta connettività di rete, è stata raggiunta l'applicazione **DVWA** dalla macchina Kali Linux ed è stato configurato il livello di sicurezza tramite la sezione **DVWA Security**.

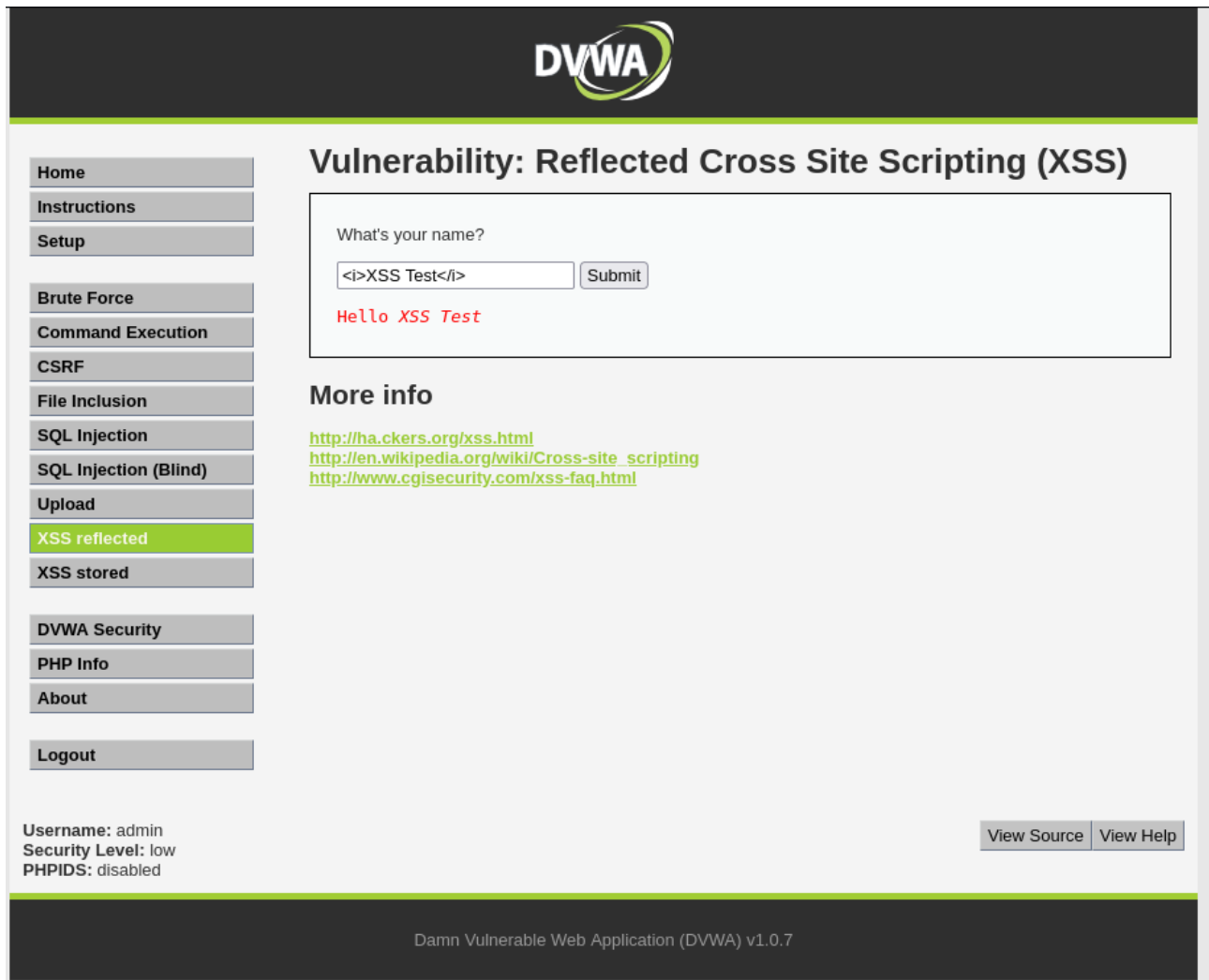
Il livello di sicurezza è stato impostato su **LOW**, mentre il sistema **PHPIDS** risulta disabilitato.

Dettagli rilevanti:

- Security Level: **LOW**
- Utente autenticato: **admin**

- PHPIDS: **disabled**

Questa configurazione riduce al minimo i controlli di validazione e sanitizzazione dell'input, rendendo l'applicazione volutamente vulnerabile e permettendo di sfruttare le vulnerabilità di **XSS Reflected** e **SQL Injection non blind** previste dall'esercizio.



The screenshot shows the DVWA web application interface. At the top is the DVWA logo. On the left is a sidebar menu with various security exercises: Home, Instructions, Setup, Brute Force, Command Execution, CSRF, File Inclusion, SQL Injection, SQL Injection (Blind), Upload, XSS reflected (highlighted in green), XSS stored, DVWA Security, PHP Info, About, and Logout. The main content area is titled "Vulnerability: Reflected Cross Site Scripting (XSS)". It contains a form with the label "What's your name?", an input field containing the payload "<i>XSS Test</i>", and a "Submit" button. Below the form, the output is displayed as "Hello *XSS Test*". Under the "More info" section, there are three links: <http://ha.ckers.org/xss.html>, http://en.wikipedia.org/wiki/Cross-site_scripting, and <http://www.cgisecurity.com/xss-faq.html>. At the bottom left, the user information is shown: "Username: admin", "Security Level: low", and "PHPIDS: disabled". At the bottom right, there are "View Source" and "View Help" buttons. The footer at the very bottom reads "Damn Vulnerable Web Application (DVWA) v1.0.7".

Accedendo alla sezione **DVWA → XSS Reflected**, è stato inserito nel campo di input il seguente payload HTML:

```
<i>XSS Test</i>
```

Analisi:

Il payload viene riflesso direttamente nella risposta HTML senza alcuna sanitizzazione o codifica dell'output. Il tag `<i>` viene interpretato correttamente dal browser, mostrando il testo in corsivo.

Risultato:

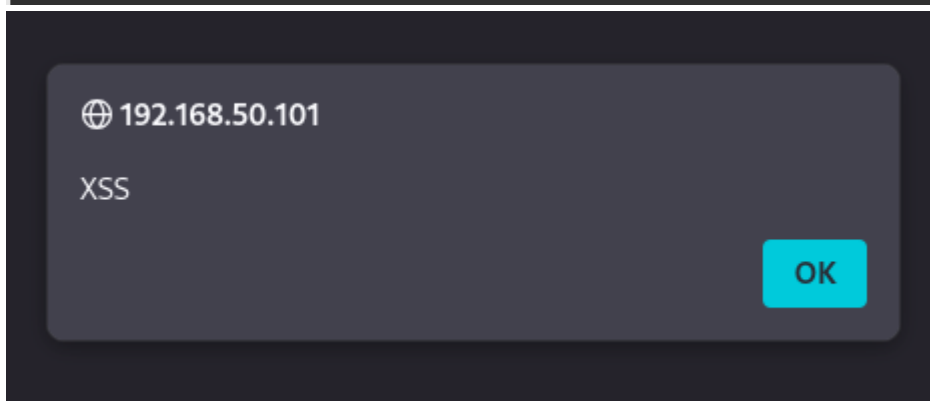
Il testo "XSS Test" viene visualizzato in corsivo all'interno della pagina di risposta, dimostrando che l'applicazione:

- riflette l'input utente senza filtri
- è vulnerabile a **HTML Injection**

- rappresenta un punto di ingresso per attacchi di **Cross-Site Scripting (XSS)**

Questo comportamento conferma la presenza di una vulnerabilità di **XSS Reflected** sfruttabile.

The screenshot shows the DVWA web application interface. At the top is the DVWA logo. On the left is a sidebar menu with various security challenges: Home, Instructions, Setup, Brute Force, Command Execution, CSRF, File Inclusion, SQL Injection, SQL Injection (Blind), Upload, XSS reflected (highlighted in green), XSS stored, DVWA Security, PHP Info, About, and Logout. The main content area is titled 'Vulnerability: Reflected Cross Site Scripting (XSS)'. It contains a form with the text 'What's your name?' and an input field containing the payload '<script>alert('XSS')</script>'. A 'Submit' button is next to the input. Below the input, the word 'Hello' is displayed in red, indicating the successful execution of the script. Under the 'More info' section, there are three links: <http://ha.ckers.org/xss.html>, http://en.wikipedia.org/wiki/Cross-site_scripting, and <http://www.cgisecurity.com/xss-faq.html>. At the bottom left, the user information is shown: 'Username: admin', 'Security Level: low', and 'PHPIDS: disabled'. At the bottom right, there are 'View Source' and 'View Help' buttons. The footer of the application states 'Damn Vulnerable Web Application (DVWA) v1.0.7'.



Nella sezione **DVWA → XSS Reflected**, è stato inserito nel campo di input il seguente payload JavaScript:

```
<script>alert('XSS')</script>
```

Analisi:

Il payload viene riflesso direttamente nella risposta dell'applicazione senza alcuna sanitizzazione o encoding dell'output.

Il browser interpreta il tag <script> ed esegue il codice JavaScript contenuto al suo interno.

Risultato:

Alla sottomissione del form, viene visualizzato un popup di tipo alert con il messaggio **"XSS"**, confermando:

- l'esecuzione di codice JavaScript arbitrario lato client
- la presenza di una vulnerabilità di **Cross-Site Scripting Reflected**
- la possibilità di sfruttare l'input utente per attacchi più avanzati (es. furto di cookie, session hijacking)

Questa prova dimostra come un attaccante possa iniettare ed eseguire script nel browser della vittima semplicemente inducendola a visitare un URL o a inviare un input malevolo.



- Home
- Instructions
- Setup
- Brute Force
- Command Execution
- CSRF
- File Inclusion
- SQL Injection
- SQL Injection (Blind)
- Upload
- XSS reflected**
- XSS stored
- DVWA Security
- PHP Info
- About
- Logout

Vulnerability: Reflected Cross Site Scripting (XSS)

What's your name?

Hello

More info

<http://ha.ckers.org/xss.html>
http://en.wikipedia.org/wiki/Cross-site_scripting
<http://www.cgisecurity.com/xss-faq.html>

Username: admin
 Security Level: low
 PHPIDS: disabled

Damn Vulnerable Web Application (DVWA) v1.0.7

🌐 192.168.50.101

security=low;
 PHPSESSID=439622e56561306d1a980642c9f2b6e7

OK

Nella sezione **DVWA → XSS Reflected**, è stato inserito un payload JavaScript volto a recuperare i cookie di sessione del browser tramite l'oggetto `document.cookie`.

Payload utilizzato:

```
<script>alert(document.cookie)</script>
```

Analisi:

Il codice JavaScript iniettato viene riflesso ed eseguito senza alcuna sanitizzazione.

L'uso di document.cookie consente di leggere i cookie associati al dominio dell'applicazione, inclusi quelli di sessione.

Risultato:

Alla sottomissione del form, viene visualizzato un popup contenente i cookie di sessione, tra cui:

- PHPSESSID
- valore del livello di sicurezza (security=low)

Questo dimostra che:

- i cookie non sono protetti tramite flag **HttpOnly**
- un attaccante potrebbe sottrarre la sessione dell'utente
- la vulnerabilità XSS può portare a **session hijacking**

Il recupero dei cookie conferma l'elevato impatto della vulnerabilità di **XSS Reflected**, che può compromettere l'autenticazione e la sicurezza dell'utente.

The screenshot shows the DVWA web application interface. At the top is the DVWA logo. On the left is a sidebar menu with various security topics. The main content area is titled 'Vulnerability: SQL Injection' and contains a 'User ID:' form with a 'Submit' button. Below the form, five rows of user data are displayed, each corresponding to a different SQL injection payload. At the bottom, there are links for more information and buttons for 'View Source' and 'View Help'.

DVWA

Vulnerability: SQL Injection

User ID:

ID: 1 ' OR '1' = '1
First name: admin
Surname: admin

ID: 1 ' OR '1' = '1
First name: Gordon
Surname: Brown

ID: 1 ' OR '1' = '1
First name: Hack
Surname: Me

ID: 1 ' OR '1' = '1
First name: Pablo
Surname: Picasso

ID: 1 ' OR '1' = '1
First name: Bob
Surname: Smith

More info

<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
http://en.wikipedia.org/wiki/SQL_injection
<http://www.unixwiz.net/techtips/sql-injection.html>

Username: admin
Security Level: low
PHPIDS: disabled

Accedendo alla sezione **DVWA → SQL Injection**, con livello di sicurezza impostato su **LOW**, è stato inserito nel campo *User ID* il seguente payload SQL:

```
1' OR '1'='1
```

Analisi:

Il payload altera la logica della query SQL originale inserendo una condizione sempre vera (OR '1'='1'). In assenza di sanitizzazione dell'input e di prepared statements, il database interpreta il comando in modo diretto, eseguendo una query modificata dall'utente.

Risultato:

L'applicazione restituisce **tutti i record presenti nella tabella degli utenti**, mostrando più risultati con nomi e cognomi differenti.

Questo comportamento conferma:

- la presenza di una **SQL Injection non blind**
- la possibilità di bypassare i controlli logici previsti dall'applicazione
- l'esposizione diretta dei dati del database all'utente non autorizzato



- Home
- Instructions
- Setup
- Brute Force
- Command Execution
- CSRF
- File Inclusion
- SQL Injection
- SQL Injection (Blind)
- Upload
- XSS reflected
- XSS stored
- DVWA Security
- PHP Info
- About
- Logout

Vulnerability: SQL Injection

User ID:


```

ID: 1' UNION SELECT user, password FROM users-- -
First name: admin
Surname: admin

ID: 1' UNION SELECT user, password FROM users-- -
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: 1' UNION SELECT user, password FROM users-- -
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03

ID: 1' UNION SELECT user, password FROM users-- -
First name: 1337
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b

ID: 1' UNION SELECT user, password FROM users-- -
First name: pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7

ID: 1' UNION SELECT user, password FROM users-- -
First name: smithy
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

```

More info

<http://www.securiteam.com/securityreviews/5DP0N1P76E.html>
http://en.wikipedia.org/wiki/SQL_injection
<http://www.unixwiz.net/techtips/sql-injection.html>

Username: admin
 Security Level: low
 PHP Info: disabled

Dopo aver confermato la presenza della vulnerabilità di SQL Injection, è stata sfruttata una **UNION-based SQL Injection** per estrarre informazioni sensibili dal database.

Nel campo *User ID* è stato inserito il seguente payload SQL:

1' UNION SELECT user, password FROM users-- -

Analisi:

La clausola UNION SELECT consente di combinare il risultato della query originale con una seconda query arbitraria controllata dall'attaccante. In assenza di prepared statements e di controlli sull'input, il database esegue la query aggiuntiva permettendo l'accesso diretto ai dati contenuti nella tabella users.

Risultato:

L'applicazione restituisce:

- nomi utente
- hash delle password associate agli account

Questo dimostra che un attaccante può:

- estrarre credenziali dal database
- compromettere la confidenzialità dei dati
- effettuare successivi attacchi di password cracking offline

La riuscita della UNION-based SQL Injection conferma l'elevata criticità della vulnerabilità.

Conclusione

L'attività di laboratorio ha permesso di individuare e sfruttare con successo due vulnerabilità web critiche presenti nell'applicazione **DVWA**, configurata con livello di sicurezza **LOW**: **Cross-Site Scripting (XSS) Reflected** e **SQL Injection non blind**.

Attraverso la vulnerabilità di **XSS Reflected**, è stato possibile iniettare ed eseguire codice JavaScript arbitrario nel browser dell'utente, dimostrando il potenziale rischio di furto dei cookie di sessione e di compromissione dell'autenticazione.

Questo evidenzia come la mancata sanitizzazione dell'input e dell'output possa portare a gravi impatti lato client, inclusi attacchi di session hijacking.

Parallelamente, la vulnerabilità di **SQL Injection non blind** ha consentito di manipolare le query SQL lato server, permettendo prima il bypass dei controlli logici e successivamente l'estrazione di dati sensibili dal database tramite tecniche UNION-based.

La possibilità di recuperare nomi utente e hash delle password rappresenta un rischio critico per la confidenzialità e l'integrità dei dati applicativi.

L'esercizio dimostra come l'assenza di controlli adeguati sull'input utente, l'uso di query SQL dinamiche e la mancata applicazione di meccanismi di sicurezza fondamentali possano portare alla compromissione completa di un'applicazione web.

Queste vulnerabilità, se presenti in un contesto reale, potrebbero essere facilmente sfruttate da un attaccante per ottenere accesso non autorizzato, esfiltrare dati sensibili e compromettere l'infrastruttura sottostante.

Venezia, 18/12/2025

Fabio Renna